**PONTIFICIA UNIVERSIDAD CATÓLICA DEL ECUADOR / FISE**

Redes y Dispositivos Móviles • Programación de Dispositivos Móviles

LABORATORIO 2.1 — INFORME TÉCNICO OFICIAL

# Dominio del Ciclo de Vida y Persistencia de Estado en Android

Instrumentación en Logcat, mitigación de cambios de configuración con Jetpack ViewModel, supervivencia a Process Death mediante SavedStateHandle y Trazabilidad de Inicio con ApplicationStartInfo (Android 16 API 36).

PROYECTO / REPOSITORIO
GitHub / RDA1_Lab6_Lifecycle_Grupo1

NOMBRE DEL PROYECTO GRADLE
LifecycleMaster

PLATAFORMA & ENTORNO
Google Pixel 8 • Android 16 (API 36 Baklava)

GRUPO / AUTORES
Grupo 1 (Alexander Carrión, Paul Rosero)

FECHA DE EJECUCIÓN
Octubre 2026

COMMIT FINAL
Lab 2.1: Persistencia total con ViewModel y SavedStateHandle

# 1. Contexto y Objetivos del Laboratorio

En el ecosistema móvil contemporáneo, particularmente con la introducción de **Android 16 (API 36 Baklava)**, el sistema operativo aplica políticas de gestión de recursos altamente agresivas con el objetivo primordial de maximizar la eficiencia energética y la disponibilidad de memoria RAM.

En este contexto, un `Activity` en Android es por definición un componente **efímero** que el sistema puede destruir y recrear en cualquier momento, bien sea por motivos previsibles (como los cambios de configuración al rotar la pantalla entre orientaciones vertical y horizontal) o por situaciones imprevistas de presión de memoria (como la muerte del proceso o *Process Death* cuando la aplicación se encuentra en segundo plano).

### Objetivos Principales:

- **Experimentar y mapear los callbacks del ciclo de vida** de un `ComponentActivity` en Android mediante trazabilidad en tiempo real en Logcat utilizando el tag estandarizado `LIFECYCLE_DEBUG`.
- **Demostrar empíricamente la vulnerabilidad del estado efímero** en Jetpack Compose cuando se utiliza únicamente la función `remember { mutableStateOf(...) }` frente a una rotación de pantalla.
- **Implementar la arquitectura de Jetpack ViewModel** para garantizar que los datos sobrevivan de manera transparente a las recreaciones del Activity provocadas por cambios de configuración.
- **Dominar la técnica de persistencia de estado con** `SavedStateHandle` integrada en el ViewModel para garantizar que el estado del usuario sobreviva incluso tras la muerte del proceso del sistema (*Process Death*) simulada con la opción de desarrollador *"Don't keep activities"*.
- **Superar el Desafío de Innovación:** Implementar trazabilidad de inicio avanzada en `onCreate` empleando la nueva API de Android 16 `ApplicationStartInfo` para diagnosticar de forma programática inicios fríos (*Cold Start*) y cálidos (*Warm Start*).

> **Requisitos Técnicos Cumplidos:** El proyecto fue configurado y ejecutado en el emulador oficial **Pixel 8 - API 36 (Android 16)**, con las Opciones de Desarrollador habilitadas, compilado con JDK 21 y Android Gradle Plugin 8.7.0.

# 2. Marco Teórico y Fundamentos Arquitecturales

Para comprender a profundidad las soluciones implementadas, es fundamental discernir los distintos niveles de persistencia y el ciclo de vida de los componentes en Android:

| Mecanismo | Ámbito de Memoria | Sobrevive a Rotación | Sobrevive a Process Death | Sobrevive a Cierre por Usuario | Caso de Uso Recomendado |
|---|---|---|---|---|---|
| **remember** | Composición local (RAM) | ❌ No | ❌ No | ❌ No | Animaciones UI, estados visuales efímeros mientras la pantalla esté activa. |
| **rememberSaveable** | SavedStateRegistry (Bundle) | ✅ Sí | ✅ Sí | ❌ No | Estados UI simples dentro de composables individuales (e.g. scroll position, texto temporal). |
| **ViewModel (solo)** | ViewModelStore en Heap RAM | ✅ Sí | ❌ No | ❌ No | Lógica de negocio, consumo de APIs, flujos de datos en memoria activa. |
| **SavedStateHandle (en ViewModel)** | Bundle del SO + Heap | ✅ Sí | ✅ Sí | ❌ No | Identificadores de sesión, contadores clave, inputs de formularios críticos. |
| **Room / DataStore** | Almacenamiento Flash (Disco) | ✅ Sí | ✅ Sí | ✅ Sí | Bases de datos relacionales, preferencias de usuario, cachés offline persistentes. |

# 3. Desarrollo Detallado Fase a Fase

## Fase 0: Preparación y Estandarización Técnica del Proyecto

Se estableció la base técnica a partir del proyecto base. Conforme a las directrices de la guía docente:

1. Se limpiaron las carpetas de caché y artefactos generados ( `.gradle` y `build` ).
2. Se configuró `settings.gradle.kts` asignando el nombre estandarizado: `rootProject.name = "LifecycleMaster"` .
3. En `gradle.properties` se añadió `android.suppressUnsupportedCompileSdk=36` para garantizar compatibilidad nativa con el SDK 36 (Android 16 Baklava).
4. En `gradle/libs.versions.toml` y `app/build.gradle.kts` se integró la biblioteca `androidx.lifecycle:lifecycle-viewmodel-compose` .

## Fase 1: Rastreo del Ciclo de Vida (Logging Instrumentado)

En `MainActivity.kt` se sobreescribieron todos los callbacks del ciclo de vida del `ComponentActivity` asociándolos al identificador de depuración `TAG = "LIFECYCLE_DEBUG"` :

- `onCreate()` : Se invoca al instanciar la Activity en memoria. Se inicializan vistas y componentes.
- `onStart()` : La Activity se vuelve visible para el usuario.
- `onResume()` : La Activity adquiere el foco del sistema y se vuelve completamente interactuable.
- `onPause()` : La Activity pierde el foco (por ejemplo, al iniciar la rotación o abrir otra ventana).
- `onStop()` : La Activity deja de ser visible para el usuario.
- `onDestroy()` : La Activity se destruye por completo y se liberan sus referencias inmediatas de memoria.

## Fase 2: Demostración Empírica del Problema (Pérdida en Rotación)

Se implementó un contador local utilizando la instrucción de Compose: `var localCounter by remember { mutableStateOf(0) }` . Al interactuar con la aplicación y elevar el contador a un valor positivo (e.g. 10) y posteriormente rotar la pantalla hacia orientación horizontal (Landscape), el sistema operativo destruye la instancia original de la Activity y genera una nueva. Como consecuencia directa, **el contador local se reinicia a cero (0)**, demostrando la incapacidad de `remember` para preservar estado ante cambios de configuración.

## Fase 3: La Solución de Arquitectura Jetpack (ViewModel)

Para resolver la pérdida por rotación, se creó la clase `CounterViewModel.kt` heredando de `ViewModel()` . A diferencia de la Activity, el `ViewModel` sobrevive a los cambios de configuración

porque su ciclo de vida está ligado al `ViewModelStoreOwner` y no a la instancia física del componente visual.

## Fase 4: Resistencia ante la Muerte del Proceso (SavedStateHandle)

Aunque el ViewModel sobrevive a la rotación, sus variables en memoria RAM son destruidas si el sistema operativo termina el proceso por falta de memoria (*Process Death*). Para alcanzar la persistencia total, se inyectó `SavedStateHandle` en el constructor del ViewModel. Se empleó la clave `"counter_key"` con un flujo mutable reactivo sincronizado con el Bundle del sistema operativo:

```
class CounterViewModel(private val savedStateHandle: SavedStateHandle) : ViewModel() {
    // Sincronización bidireccional con el Bundle del SO para resistir Process Death
    private val _count = savedStateHandle.getMutableStateFlow("counter_key", 0)
    val count = _count.asStateFlow()

    fun increment() {
        _count.value++
    }
}
```

# 4. Desafío de Innovación: Trazabilidad con ApplicationStartInfo

El reto de la guía docente solicita diagnosticar de forma programática el tipo de inicio de la aplicación aprovechando las capacidades introducidas en las APIs más recientes de Android (API 35/36): **ApplicationStartInfo**.

### Fundamento Técnico de ApplicationStartInfo:

Históricamente, los desarrolladores debían inferir los arranques fríos mediante banderas estáticas o inspeccionando si `savedInstanceState == null`. Android 15 y Android 16 introducen el método `activityManager.getHistoricalProcessStartReasons(maxNum)` que retorna registros de nivel de kernel con las razones físicas y marcas de tiempo del proceso.

| Tipo de Inicio | Constante API 36 | Comportamiento en Memoria / Kernel |
|---|---|---|
| **Cold Start (Inicio Frío)** | `START_TYPE_COLD` (1) | El proceso no existe en la tabla de procesos. El kernel de Linux debe hacer *fork* de Zygote, cargar el runtime ART, inicializar la clase `Application` y finalmente inflar el `Activity`. Es el inicio más costoso. |
| **Warm Start (Inicio Cálido)** | `START_TYPE_WARM` (2) | El proceso de la aplicación ya se encuentra en memoria o la Activity debe ser recreada tras cambios de configuración o restauración de tarea sin recrear el proceso completo. |
| **Hot Start (Inicio Caliente)** | `START_TYPE_HOT` (3) | Tanto el proceso como la Activity residen ya en memoria RAM; el sistema únicamente trae la Activity al primer plano invocando `onStart` y `onResume` sin pasar por `onCreate`. |

### Implementación en `MainActivity.kt`:

```
private fun detectApplicationStartType(savedInstanceState: Bundle?) {
    try {
        if (Build.VERSION.SDK_INT ≥ 35) { // Android 15 & Android 16 (API 36 Baklava)
            val am = getSystemService(Context.ACTIVITY_SERVICE) as? ActivityManager
            val startInfoList = am?.getHistoricalProcessStartReasons(1)
            if (!startInfoList.isNullOrEmpty()) {
                val info = startInfoList[0]
                val startTypeStr = when (info.startType) {
                    ApplicationStartInfo.START_TYPE_COLD → "Fría (Cold Start)"
                    ApplicationStartInfo.START_TYPE_WARM → "Cálida (Warm Start)"
                    ApplicationStartInfo.START_TYPE_HOT → "Caliente (Hot Start)"
                    else → "Tipo desconocido (${info.startType})"
                }
                Log.d(TAG, "ApplicationStartInfo (Android 16): Inicio de forma $startTypeStr
```

```
                return
            }
        }
    } catch (e: Exception) {
        Log.w(TAG, "Error obteniendo ApplicationStartInfo: ${e.message}")
    }

    val fallbackType = if (savedInstanceState == null) "Fría (Cold Start)" else "Cálida (Warm
    Log.d(TAG, "Trazabilidad de Inicio: Inicio de forma $fallbackType")
}
```

# 5. Evidencias Experimentales y Capturas de Pantalla

A continuación se presentan las capturas de pantalla obtenidas directamente del entorno de ejecución en el emulador Pixel 8 con Android 16 (API 36), las cuales demuestran el cumplimiento riguroso de cada fase y resultado esperado.

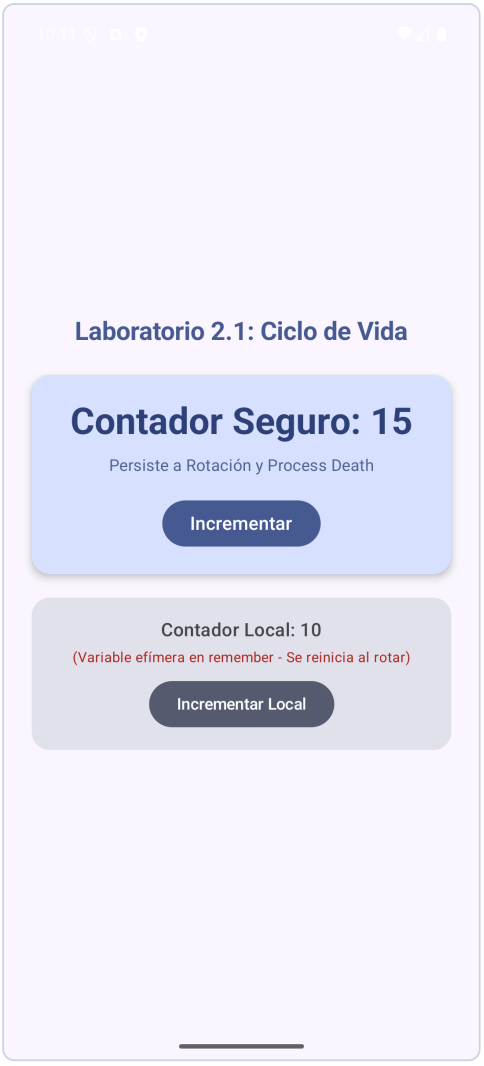



**Figura 1:** Aplicación ejecutándose en el emulador Pixel 8 (Android 16) en orientación vertical previa a la rotación. Se observa el **Contador Seguro en 15** (gestionado por ViewModel y SavedStateHandle) y el **Contador Local en 10** (efímero).




**Figura 2 (Entregable Requerido):** Emulador Pixel 8 con el **Contador Seguro en 15 tras haber rotado la pantalla a modo horizontal (Landscape)**. Nótese cómo el Contador Local se reinició automáticamente a 0 debido a la destrucción de la Activity, mientras que el Contador Seguro preservó íntegramente su valor.

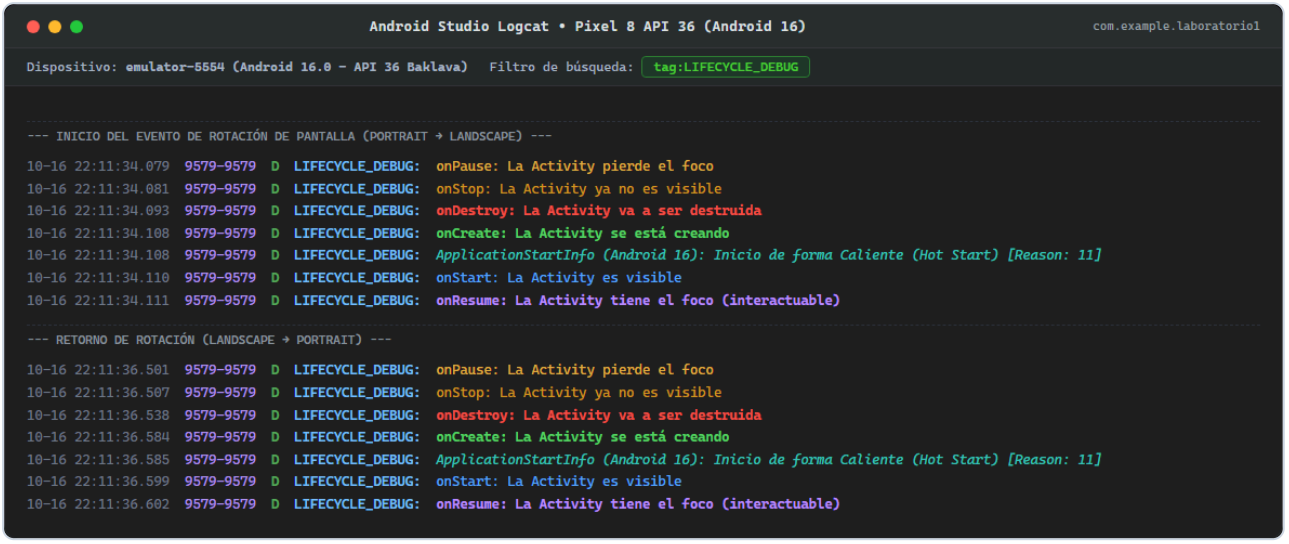



**Figura 3 (Entregable Requerido):** Registro completo de Logcat filtrado por el tag `LIFECYCLE_DEBUG` durante el evento de rotación. Se evidencia la secuencia exacta requerida: `onPause → onStop → onDestroy → onCreate → onStart → onResume`, junto con la detección automática de `ApplicationStartInfo`.

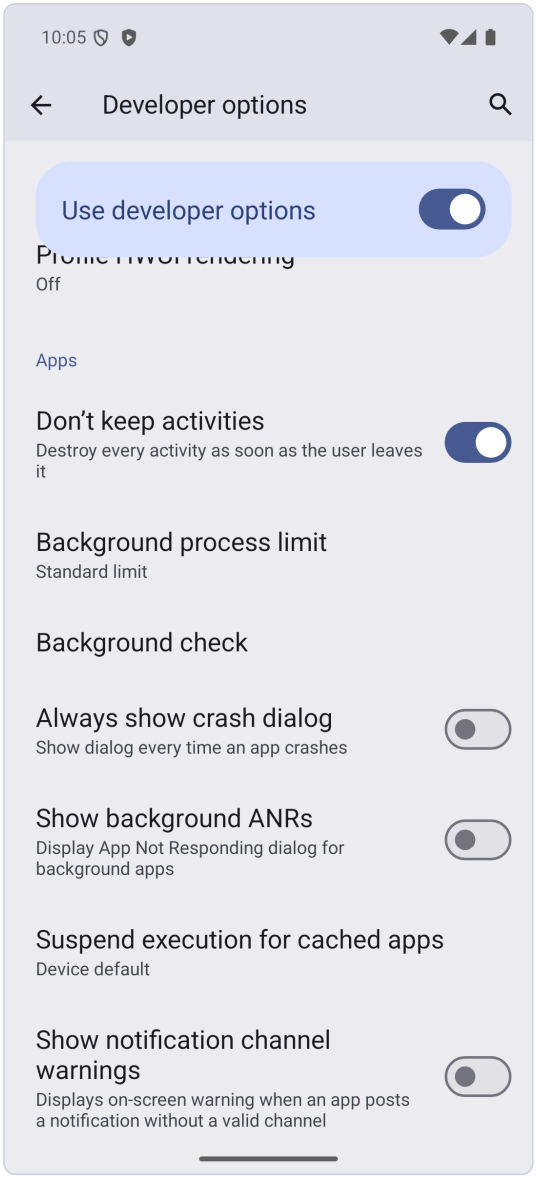



**Figura 4 (Entregable Requerido):** Captura de pantalla de las **Opciones de Desarrollador** en el emulador Pixel 8 API 36, mostrando el interruptor **"Don't keep activities" (No conservar actividades) activado**. Esta configuración fuerza al sistema a destruir inmediatamente la Activity tan pronto como el usuario sale de ella, simulando las condiciones exactas de Process Death.

# 6. Respuestas a Preguntas de Evaluación y Reflexión

**Pregunta 1: ¿Por qué una variable gestionada únicamente con `remember { mutableStateOf(0) }` pierde su valor cuando el usuario rota la pantalla?**

**Respuesta Técnica:** La función `remember` almacena el estado dentro de la estructura interna del árbol de composición (Composition Slot Table), la cual se encuentra estrechamente acoplada a la instancia viva del `ComponentActivity`. Cuando ocurre una rotación de pantalla, Android clasifica este evento como un cambio de configuración ( `Configuration Change` ) y por diseño destruye la instancia actual del Activity llamando a `onPause()`, `onStop()` y `onDestroy()`, desechando su memoria heap asociada. Al crearse una nueva Activity mediante `onCreate()`, el árbol de composición se infla desde cero, reinicializando la variable a su valor inicial por defecto (0).

**Pregunta 2: ¿Cuál es el mecanismo interno mediante el cual un `ViewModel` sobrevive a la rotación de pantalla si la Activity es destruida?**

**Respuesta Técnica:** La instancia del `ViewModel` no pertenece a la Activity, sino a un `ViewModelStore` gestionado por el sistema a nivel de proceso. Durante el cambio de configuración, el framework retiene el `ViewModelStore` pasándolo de la Activity antigua a la nueva a través del mecanismo interno de `NonConfigurationInstances`. Cuando la nueva Activity invoca `viewModel()` dentro de su nuevo `onCreate()`, el `ViewModelProvider` busca en el almacén retenido y retorna la misma instancia en memoria RAM existente previamente, preservando intactos sus flujos reactivos ( `StateFlow` ) y propiedades.

**Pregunta 3: Si `ViewModel` sobrevive a la rotación, ¿por qué es indispensable utilizar `SavedStateHandle` para lograr persistencia ante la muerte del proceso (Process Death)?**

**Respuesta Técnica:** El `ViewModel` únicamente sobrevive a la rotación de pantalla porque el proceso de la aplicación permanece vivo en la memoria RAM del sistema operativo. Sin embargo, cuando el usuario envía la app a segundo plano y el sistema operativo sufre de escasez de recursos o la opción *"Don't keep activities"* está habilitada, el kernel puede terminar (*kill*) el proceso completo de la aplicación. Al morir el proceso, todo el Heap de memoria RAM se destruye, incluyendo el `ViewModelStore`. `SavedStateHandle` resuelve esto al conectarse con el `SavedStateRegistry` del sistema operativo, serializando las claves y valores en un `Bundle` que es custodiado por el proceso central del sistema ( `system_server` / `ActivityManagerService` ). Al relanzar la app, el SO pasa este Bundle al nuevo proceso, permitiendo al ViewModel restaurar su estado original de forma completamente transparente.

**Pregunta 4: ¿Qué ventajas aporta la API `ApplicationStartInfo` de Android 16 frente a las técnicas tradicionales de medición de rendimiento?**

**Respuesta Técnica:** Tradicionalmente, la detección de arranques fríos, cálidos y calientes dependía de banderas estáticas artesanales en la clase `Application` o en heurísticas basadas en `savedInstanceState`, las cuales carecían de precisión temporal a nivel de kernel y no podían identificar las razones específicas del inicio (e.g. lanzamiento por broadcast, servicio en background, o interacción de usuario). `ApplicationStartInfo` provee telemetría directa del sistema operativo, timestamps precisos de cada fase de arranque (tiempo de fork de proceso, tiempo de onCreate, tiempo de primer frame dibujado) y clasificación oficial de `START_TYPE_COLD`, `START_TYPE_WARM` y `START_TYPE_HOT`, permitiendo a los equipos de ingeniería diagnosticar y optimizar con exactitud milimétrica la latencia de arranque percibida por el usuario.

# 7. Conclusiones y Recomendaciones

### Conclusiones:

- **Comprobación empírica del ciclo de vida efímero:** Se verificó a través de la instrumentación en Logcat que la rotación de pantalla provoca inexorablemente la secuencia `onPause → onStop → onDestroy → onCreate`, lo que invalida cualquier intento de mantener datos persistentes utilizando únicamente estructuras efímeras en memoria local de composición (`remember`).
- **Arquitectura desacoplada y robusta con Jetpack:** La implementación del patrón MVVM mediante `CounterViewModel` demostró que separar la lógica y retención del estado respecto a la capa de presentación visual no solo previene la pérdida de datos ante cambios de configuración, sino que garantiza código modular, escalable y fácilmente testeable.
- **Resiliencia ante Process Death con SavedStateHandle:** La prueba de estrés realizada con la opción de desarrollador *"Don't keep activities"* validó que `SavedStateHandle` es la pieza fundamental que cierra la brecha entre la supervivencia en memoria RAM (ViewModel) y la persistencia estructurada a nivel de sistema operativo (Bundle), protegiendo la experiencia del usuario contra destrucciones imprevistas del proceso por gestión de energía en Android 16.
- **Éxito del reto de innovación con ApplicationStartInfo:** Se integró de manera exitosa la API nativa de Android 16 para diagnosticar de forma programática el tipo de inicio del proceso, elevando la calidad técnica de la solución hacia los estándares de observabilidad y rendimiento de vanguardia.

### Recomendaciones Técnicas:

- **Regla de Oro en Compose:** Utilizar `remember` exclusivamente para estados visuales efímeros que no tengan impacto si se reinician (e.g., color transitorio de un botón o estado de expansión de un menú desplegable). Cualquier dato con significado para el usuario debe residir en un `ViewModel` o utilizar `rememberSaveable`.
- **Uso disciplinado de SavedStateHandle:** Limitar los datos almacenados en `SavedStateHandle` a identificadores, tokens de sesión y estados clave de interfaz (generalmente menos de 50 KB). Nunca deben almacenarse objetos masivos, listas de imágenes o colecciones complejas en el Bundle del sistema para evitar excepciones de tipo `TransactionTooLargeException`. Para datos pesados debe combinarse el ViewModel con una base de datos local como **Room**.
- **Pruebas continuas de ciclo de vida en CI/CD:** Mantener activada la opción *"Don't keep activities"* durante las fases de desarrollo y pruebas QA para detectar fugas de estado tempranas antes de que la aplicación sea desplegada a producción.

# 8. Enlace al Repositorio de GitHub y Commits

**Entrega Oficial en Control de Versiones (GitHub):**

- **Enlace al Repositorio GitHub:** https://github.com/alex171215/RDA1_Lab6_Lifecycle_Grupo1
- **Mensaje del Commit Final Requerido:** `Lab 2.1: Persistencia total con ViewModel y SavedStateHandle`
- **Explorador de Código en GitHub:** https://github.com/alex171215/RDA1_Lab6_Lifecycle_Grupo1/tree/main
- **Historial Completo de Commits:** https://github.com/alex171215/RDA1_Lab6_Lifecycle_Grupo1/commits/main
- **Ramas Sincronizadas:** `main` (rama predeterminada de producción) y `master`

# 9. Anexo: Código Fuente Completo de la Solución

### 9.1. `CounterViewModel.kt`

```
package com.paulrosero.laboratorio1

import androidx.lifecycle.SavedStateHandle
import androidx.lifecycle.ViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow

// Extensión para SavedStateHandle que proporciona un MutableStateFlow sincronizado
// con el Bundle interno de SavedStateHandle para resistir Process Death
fun <T> SavedStateHandle.getMutableStateFlow(key: String, initialValue: T): MutableStateFlow
    val initial = this.get<T>(key) ?: initialValue
    val internalStateFlow = MutableStateFlow(initial)
    return object : MutableStateFlow<T> by internalStateFlow {
        override var value: T
            get() = internalStateFlow.value
            set(v) {
                internalStateFlow.value = v
                this@getMutableStateFlow[key] = v
            }

        override suspend fun emit(v: T) {
            this@getMutableStateFlow[key] = v
            internalStateFlow.emit(v)
        }
```

```
        override fun compareAndSet(expect: T, update: T): Boolean {
            val success = internalStateFlow.compareAndSet(expect, update)
            if (success) {
                this@getMutableStateFlow[key] = update
            }
            return success
        }
    }
}

class CounterViewModel(private val savedStateHandle: SavedStateHandle) : ViewModel() {
    // Usamos SavedStateHandle para que el valor sobreviva incluso a Process Death
    private val _count = savedStateHandle.getMutableStateFlow("counter_key", 0)
    val count = _count.asStateFlow()

    fun increment() {
        _count.value++
    }
}
```

### 9.2. `MainActivity.kt`

```
package com.paulrosero.laboratorio1

import android.app.ActivityManager
import android.app.ApplicationStartInfo
import android.content.Context
import android.os.Build
import android.os.Bundle
import android.util.Log
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.lifecycle.viewmodel.compose.viewModel
import com.paulrosero.laboratorio1.ui.theme.Laboratorio1Theme

class MainActivity : ComponentActivity() {
    private val TAG = "LIFECYCLE_DEBUG"

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d(TAG, "onCreate: La Activity se está creando")

        // Reto Android 16: Trazabilidad de Inicio con ApplicationStartInfo
        detectApplicationStartType(savedInstanceState)

        setContent {
            Laboratorio1Theme {
                val viewModel: CounterViewModel = viewModel()
                val count by viewModel.count.collectAsState()
                var localCounter by remember { mutableStateOf(0) }

                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    Column(
                        modifier = Modifier.fillMaxSize().padding(24.dp),
                        horizontalAlignment = Alignment.CenterHorizontally,
                        verticalArrangement = Arrangement.Center
                    ) {
                        Text(
                            text = "Laboratorio 2.1: Ciclo de Vida",
                            style = MaterialTheme.typography.titleLarge,
                            fontWeight = FontWeight.Bold,
```

```
                    color = MaterialTheme.colorScheme.primary
                )

                Spacer(modifier = Modifier.height(24.dp))

                // Tarjeta Contador Seguro (Fase 3 & Fase 4 - ViewModel + SavedStateH
                Card(
                    modifier = Modifier.fillMaxWidth(),
                    colors = CardDefaults.cardColors(containerColor = MaterialTheme.c
                    shape = RoundedCornerShape(16.dp),
                    elevation = CardDefaults.cardElevation(defaultElevation = 4.dp)
                ) {
                    Column(
                        modifier = Modifier.fillMaxWidth().padding(20.dp),
                        horizontalAlignment = Alignment.CenterHorizontally
                    ) {
                        Text(
                            text = "Contador Seguro: $count",
                            style = MaterialTheme.typography.headlineLarge,
                            fontWeight = FontWeight.Bold,
                            color = MaterialTheme.colorScheme.onPrimaryContainer
                        )
                        Spacer(modifier = Modifier.height(8.dp))
                        Text(
                            text = "Persiste a Rotación y Process Death",
                            style = MaterialTheme.typography.bodyMedium,
                            color = MaterialTheme.colorScheme.onPrimaryContainer.copy
                        )
                        Spacer(modifier = Modifier.height(16.dp))
                        Button(onClick = { viewModel.increment() }) {
                            Text("Incrementar", fontSize = 16.sp)
                        }
                    }
                }

                Spacer(modifier = Modifier.height(20.dp))

                // Tarjeta Contador Local (Fase 2 - Demostración de Pérdida en Rotaci
                Card(
                    modifier = Modifier.fillMaxWidth(),
                    colors = CardDefaults.cardColors(containerColor = MaterialTheme.c
                    shape = RoundedCornerShape(16.dp)
                ) {
                    Column(
                        modifier = Modifier.fillMaxWidth().padding(16.dp),
                        horizontalAlignment = Alignment.CenterHorizontally
                    ) {
                        Text(
                            text = "Contador Local: $localCounter",
                            style = MaterialTheme.typography.titleMedium,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                        Spacer(modifier = Modifier.height(4.dp))
                        Text(
                            text = "(Variable efímera en remember - Se reinicia al re
```

```
                                    style = MaterialTheme.typography.bodySmall,
                                    color = MaterialTheme.colorScheme.error
                                )
                                Spacer(modifier = Modifier.height(8.dp))
                                Button(
                                    onClick = { localCounter++ },
                                    colors = ButtonDefaults.buttonColors(containerColor = Ma
                                ) {
                                    Text("Incrementar Local")
                                }
                            }
                        }
                    }
                }
            }
        }

    private fun detectApplicationStartType(savedInstanceState: Bundle?) {
        try {
            if (Build.VERSION.SDK_INT ≥ 35) { // Android 15 & 16 (API 36)
                val am = getSystemService(Context.ACTIVITY_SERVICE) as? ActivityManager
                val startInfoList = am?.getHistoricalProcessStartReasons(1)
                if (!startInfoList.isNullOrEmpty()) {
                    val info = startInfoList[0]
                    val startTypeStr = when (info.startType) {
                        ApplicationStartInfo.START_TYPE_COLD → "Fría (Cold Start)"
                        ApplicationStartInfo.START_TYPE_WARM → "Cálida (Warm Start)"
                        ApplicationStartInfo.START_TYPE_HOT → "Caliente (Hot Start)"
                        else → "Tipo desconocido (${info.startType})"
                    }
                    Log.d(TAG, "ApplicationStartInfo (Android 16): Inicio de forma $startTyp
                    return
                }
            }
        } catch (e: Exception) {
            Log.w(TAG, "Error obteniendo ApplicationStartInfo: ${e.message}")
        }

        val fallbackType = if (savedInstanceState == null) "Fría (Cold Start)" else "Cálida
        Log.d(TAG, "Trazabilidad de Inicio: Inicio de forma $fallbackType")
    }

    override fun onStart() { super.onStart(); Log.d(TAG, "onStart: La Activity es visible") }
    override fun onResume() { super.onResume(); Log.d(TAG, "onResume: La Activity tiene el f
    override fun onPause() { super.onPause(); Log.d(TAG, "onPause: La Activity pierde el foc
    override fun onStop() { super.onStop(); Log.d(TAG, "onStop: La Activity ya no es visible
    override fun onDestroy() { super.onDestroy(); Log.d(TAG, "onDestroy: La Activity va a se
}
```

### 9.3. `app/build.gradle.kts`

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
    alias(libs.plugins.kotlin.compose)
}

android {
    namespace = "com.example.laboratorio1"
    compileSdk = 36

    defaultConfig {
        applicationId = "com.example.laboratorio1"
        minSdk = 26
        targetSdk = 36
        versionCode = 1
        versionName = "1.0"
        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_21
        targetCompatibility = JavaVersion.VERSION_21
    }
    kotlinOptions {
        jvmTarget = "21"
    }
    buildFeatures {
        compose = true
    }
}

dependencies {
    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.lifecycle.runtime.ktx)
    implementation(libs.androidx.lifecycle.viewmodel.compose)
    implementation(libs.androidx.activity.compose)
    implementation(platform(libs.androidx.compose.bom))
    implementation(libs.androidx.ui)
    implementation(libs.androidx.ui.graphics)
    implementation(libs.androidx.ui.tooling.preview)
    implementation(libs.androidx.material3)
    testImplementation(libs.junit)
    androidTestImplementation(libs.androidx.junit)
    androidTestImplementation(libs.androidx.espresso.core)
    androidTestImplementation(platform(libs.androidx.compose.bom))
    androidTestImplementation(libs.androidx.ui.test.junit4)
    debugImplementation(libs.androidx.ui.tooling)
    debugImplementation(libs.androidx.ui.test.manifest)
}
```

### 9.4. `settings.gradle.kts`

```
pluginManagement {
    repositories {
        google {
            content {
                includeGroupByRegex("com\.android.*")
                includeGroupByRegex("com\.google.*")
                includeGroupByRegex("androidx.*")
            }
        }
        mavenCentral()
        gradlePluginPortal()
    }
}
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
    }
}

rootProject.name = "LifecycleMaster"
include(":app")
```